

Rapport de Benchmark : Infrastructure de Données Low-Latency pour Jawas

Ce document analyse quatre solutions critiques pour la réception de signaux de liquidation sur Solana : **Pyth (Hermes)**, **Triton**, **Helius**, et **Liquify**. L'objectif est d'identifier le mix technologique permettant de réduire le délai de réaction de Jawas.

1. Tableau Comparatif Synthétique

Critère	Pyth (Hermes)	Triton (Yellowstone)	Helius (Geyser)	Liquify (Geyser)
Spécialité	Prix Oracle (Pull)	Performance Pure / gRPC	DevEx / Filtrage	Fiabilité / gRPC
Précocité du signal	★★★★★ (Prix)	★★★★★ (Comptes)	★★★★☆	★★★★☆
Qualité du signal	Flux de prix uniquement	Brut (Accounts/Slots)	Très riche (Enrichi)	Brut / Standard
Stabilité (p99)	Excellente	Très Haute (Dédié)	Bonne	Très Haute
Coût Mensuel	Gratuit / Consommation	\$1,000+ (Dédié)	\$500 - \$2,000+	Sur devis (~\$1k+)
Complexité Rust	Faible (REST/WS)	Haute (gRPC/Proto)	Moyenne (SDK/gRPC)	Haute (gRPC)

2. Analyse par Critères de Performance

1. Précocité du Signal (Le "Time-to-Earliest")

- Pyth Hermes** : C'est le signal "amont". Hermes permet de récupérer le prix dès qu'il est disponible sur le réseau PythNet, *avant* qu'il ne soit poussé sur Solana (modèle Pull). C'est le signal le plus précoce pour recalculer le Health Factor.
- Triton (Yellowstone)** : Leader sur la vitesse de transmission des comptes (Account Updates). Leurs nœuds Geyser "Dragon's Mouth" sont optimisés pour réduire le jitter.

- **Helius** : Extrêmement rapide sur les logs de transactions, mais parfois un cheveu derrière Triton sur le streaming brut de comptes massifs (comme les obligations Kamino).

2. Qualité et Filtrage du Signal

- **Helius** : Offre les meilleures capacités de filtrage côté serveur. Vous pouvez ne recevoir que ce qui est strictement nécessaire pour Jawas, réduisant le bruit.
- **Triton/Liquify** : Fournissent un flux gRPC "raw". C'est à Jawas de faire le tri. Cela demande plus de puissance CPU locale mais garantit qu'aucune information n'est filtrée par erreur.
- **Capacité "Shadow State"** : Triton et Liquify sont les plus adaptés pour maintenir un état miroir complet (Shadow State) en Rust sans trous de données.

3. Stabilité Opérationnelle (Comportement sous stress)

- **Triton** : Réputé pour sa résilience lors des pics de congestion du réseau (ex: lancements de tokens majeurs). Leurs instances gRPC dédiées ne "droppent" quasiment jamais de paquets.
- **Pyth** : Très stable car découplé de la congestion RPC standard de Solana.

4. Coût de Possession (Complexité d'Intégration)

- **Pyth** : Intégration Rust très simple via des crates de désérialisation de prix.
- **Geyser (Triton/Helius/Liquify)** : Nécessite une gestion complexe des connexions gRPC, de la désérialisation Borsh en temps réel et de la gestion des re-orgs (slots non confirmés).

3. Analyse des Coûts et Accessibilité

Provider	Site Web	Coût estimé (Liquidation-ready)
Pyth Hermes	pyth.network	Gratuit (Public) ou via RPC payant.
Triton	triton.one	\$1,000 - \$2,500/mois pour un nœud Yellowstone dédié.
Helius	helius.dev	\$500/mois (Plan Pro) à \$2,000+ (Custom Geyser).
Liquify	liquify.com	\$1,000+/mois (Infrastructure institutionnelle).

4. Recommandations pour la Stack de "Jawas"

Pour battre les concurrents qui sont "plus rapides", vous ne devez pas choisir un seul fournisseur, mais **combiner les signaux** :

1. **Signal de Déclenchement (Le plus tôt possible)** : Utilisez **Pyth Hermes**. Dès qu'un prix change sur Hermes, recalculez localement la santé de vos obligations cibles. N'attendez pas que le prix soit mis à jour on-chain sur Solana.
2. **Maintien de l'État (Shadow State)** : Utilisez **Triton (Yellowstone)** via gRPC. C'est le flux le plus fiable pour savoir exactement combien de collatéral il reste dans une obligation à chaque micro-seconde.
3. **Filtrage et Logs** : **Helius** est excellent pour surveiller les transactions de la concurrence et ajuster vos priorités de frais (Priority Fees) en temps réel.

Le "Winning Move" technique en Rust :

Le secret des meilleurs bots est de maintenir une **HashMap en mémoire** de toutes les obligations Kamino à risque, mise à jour par le flux gRPC de **Triton**, et de "re-scorer" cette liste à chaque tick de **Pyth Hermes** (plusieurs fois par seconde).

Verdict final :

Si le budget est une contrainte, commencez par **Helius**. Si vous voulez la performance maximale pour gagner la course, passez sur **Triton Yellowstone** combiné à **Pyth Hermes**.